

OPERATING SYSTEMS LAB PROJECT REPORT

SRICHARAN ANAND (BT2024100)

PROBLEM STATEMENT

The project aims to design and implement a client-server-based system that simulates a co-ordinated musical performance.

In the system, the instrumentalists (clients) follow a pre-defined sequence of notes, stored as time-indexed lines of text. The server synchronises their execution by broadcasting periodic 'ticks,' like counting time with a metronome. This ensures co-ordination among performers, similar to a real-world orchestra.

The system uses role-based control, allowing an administrator to start and end a performance, which controls the server's broadcasting. Additionally, multiple clients may concurrently interact with shared files (the sheet music), requiring mechanisms to maintain data consistency. Finally, the system uses socket programming to enable communication and co-ordination between the distributed processes.

CONCEPT IMPLEMENTATION EXPLANATIONS

1) ROLE-BASED AUTHORISATION

- The system defines 2 roles: Conductor (administrator) and Instrumentalist (user).
- Administrators are responsible for managing pieces and rehearsals, while users can view and edit their parts and participate in rehearsals.

2) SOCKET PROGRAMMING

- The system uses a TCP-based client-server architecture to enable communication between the server and instrumentalists and conductor (admin).
- Admins send START and STOP requests to the server to start/stop a rehearsal.
- Clients send JOIN requests to the server to participate in a rehearsal.
- Synchronisation is achieved via a centralised timing mechanism controlled by the server.
- The server maintains multiple client connections and broadcasts synchronised ticks to all connected instrumentalists.
- The instrumentalists print the note(s) at each tick.

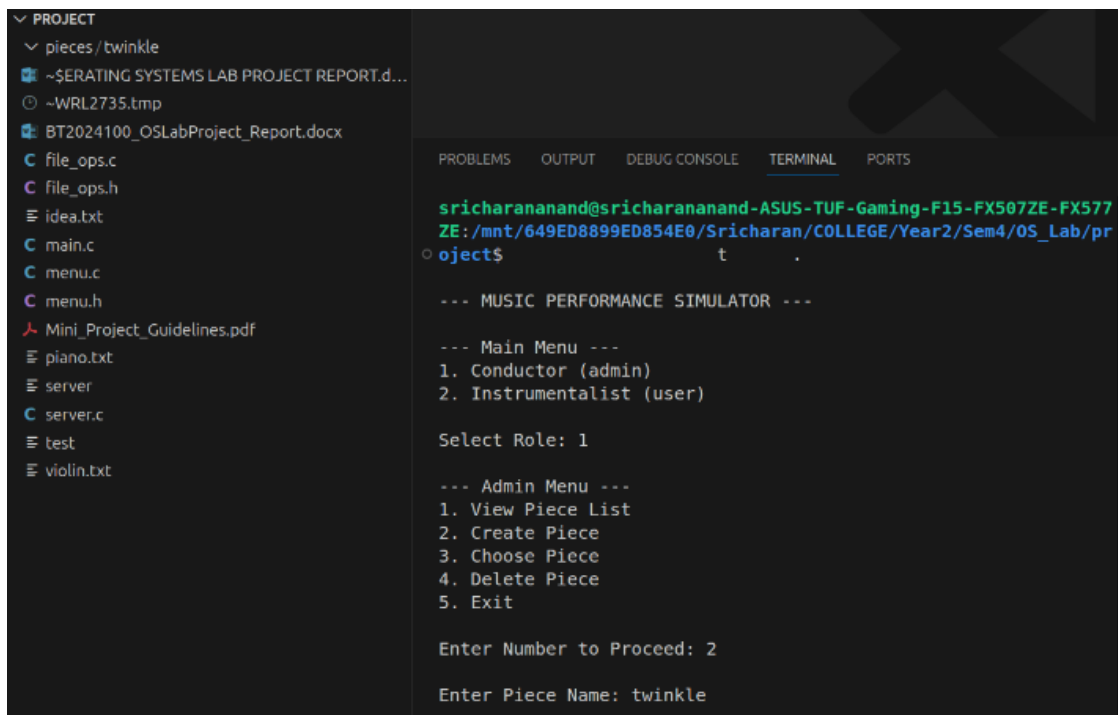
3) CONCURRENCY CONTROL & DATA CONSISTENCY

- Concurrency arises from multiple client processes accessing shared sheet music files.
- To prevent race conditions, mandatory file-level locking is implemented via the fcntl system call.
- A write lock is acquired before appending notes and released afterwards, ensuring mutual exclusion and maintaining data consistency.
- The server ensures that all clients operate on a consistent timeline and reset system state after each rehearsal.

4) IPC

- A pipe between the parent and child processes within the server is used.
- The parent process writes tick values to the pipe, while the child process reads and logs them.

SCREENSHOTS OF OUTPUT



```
PROJECT
└─ pieces/twinkle
   ~$ERATING SYSTEMS LAB PROJECT REPORT.d...
   ~WRL2735.tmp
   BT2024100_OSLabProject_Report.docx
   file_ops.c
   file_ops.h
   idea.txt
   main.c
   menu.c
   menu.h
   Mini_Project_Guidelines.pdf
   piano.txt
   server
   server.c
   test
   violin.txt

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

sricharananand@sricharananand-ASUS-TUF-Gaming-F15-FX507ZE-FX577
ZE:/mnt/649ED8899ED854E0/Sricharan/COLLEGE/Year2/Sem4/OS_Lab/pr
oject$ t .

--- MUSIC PERFORMANCE SIMULATOR ---

--- Main Menu ---
1. Conductor (admin)
2. Instrumentalist (user)

Select Role: 1

--- Admin Menu ---
1. View Piece List
2. Create Piece
3. Choose Piece
4. Delete Piece
5. Exit

Enter Number to Proceed: 2

Enter Piece Name: twinkle
```

Figure 1: Creation of Piece Directory by Administrator

```

piano.txt x
pieces > twinkle > piano.txt
1 1 C4
2

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

sricharananand@sricharananand-ASUS-TUF-Gaming-F15-FX507ZE-FX507ZE: /mnt/649ED8899ED854E0/Sricharan/COLLEGE/Year2/Sem4/OS_Lab/project$
Enter Number to Proceed: 2

Enter part name (e.g., piano.txt): piano.txt

--- twinkle ---
1. View Part List
2. Create Part
3. View Part
4. Edit Part (Append Note)
5. Delete Part
6. Start Rehearsal
7. Stop Rehearsal
8. Back

Enter Number to Proceed: 4

Parts to Choose From:
piano.txt

Enter Part Name: piano.txt

Enter Note (e.g., '3 G4'): 1 C4

```

Figure 2: Adding a Note to a Part File; file locking ensures concurrency control

<pre> ... /Sricharan/COLLEGE/Year2/Sem4 /OS_Lab/project\$./server </pre>	<pre> /OS_Lab/project\$./test 1. Conductor (admin) 2. Instrumentalist (user) Select Role: 1 --- Admin Menu --- 1. View Piece List 2. Create Piece 3. Choose Piece 4. Delete Piece 5. Exit Enter Number to Proceed: 3 Pieces to Choose From: twinkle Enter Piece Name: twinkle --- twinkle --- 1. View Part List 2. Create Part 3. View Part 4. Edit Part (Append Note) 5. Delete Part 6. Start Rehearsal 7. Stop Rehearsal 8. Back Enter Number to Proceed: 6 </pre>	<pre> /OS_Lab/project\$./test --- Main Menu --- 1. View Piece List 2. Choose Piece 3. Exit Enter Number to Proceed: 2 Pieces to Choose From: twinkle Enter Piece Name: twinkle --- twinkle --- 1. View Part List 2. View Part 3. Edit Part (Append Note) 4. Join Rehearsal 5. Back Enter Number to Proceed: 4 Parts to Choose From: piano.txt violin.txt Enter Part Name: violin.txt Ready. </pre>	<pre> /OS_Lab/project\$./test Sele2 --- Main Menu --- 1. View Piece List 2. Choose Piece 3. Exit Enter Number to Proceed: 2 Pieces to Choose From: twinkle Enter Piece Name: twinkle --- twinkle --- 1. View Part List 2. View Part 3. Edit Part (Append Note) 4. Join Rehearsal 5. Back Enter Number to Proceed: 4 Parts to Choose From: piano.txt violin.txt Enter Part Name: piano.txt Ready. </pre>
---	--	---	---

Figure 3: Ready State: Instrumentalists join rehearsal and enter ready state.

[SERVER] TICK 0 sent to 2 Instrumentalist(s) Tick received: 0 Tick received: 1 [SERVER] TICK 1 sent to 2 Instrumentalist(s) [SERVER] TICK 2 sent to 2 Instrumentalist(s) Tick received: 2 [SERVER] TICK 3 sent to 2 Instrumentalist(s) Tick received: 3 [SERVER] TICK 4 sent to 2 Instrumentalist(s) Tick received: 4 	8. Back Enter Number to Proceed: 7 --- twinkle --- 1. View Part List 2. Create Part 3. View Part 4. Edit Part (Append Note) 5. Delete Part 6. Start Rehearsal 7. Stop Rehearsal 8. Back Enter Number to Proceed: 6 --- twinkle --- 1. View Part List 2. Create Part 3. View Part 4. Edit Part (Append Note) 5. Delete Part 6. Start Rehearsal 7. Stop Rehearsal 8. Back Enter Number to Proceed:	Enter Number to Proceed: 2 Pieces to Choose From: twinkle Enter Piece Name: twinkle --- twinkle --- 1. View Part List 2. View Part 3. Edit Part (Append Note) 4. Join Rehearsal 5. Back Enter Number to Proceed: 4 Parts to Choose From: piano.txt violin.txt Enter Part Name: piano.txt Ready. [TICK 1]: C4 [TICK 2]: C4 [TICK 3]: G4 [TICK 4]: G4	3. Exit Enter Number to Proceed: 2 Pieces to Choose From: twinkle Enter Piece Name: twinkle --- twinkle --- 1. View Part List 2. View Part 3. Edit Part (Append Note) 4. Join Rehearsal 5. Back Enter Number to Proceed: 4 Parts to Choose From: piano.txt violin.txt Enter Part Name: violin.txt Ready. [TICK 1]: C4 [TICK 3]: E4
---	---	---	---

Figure 4: Synchronised Tick Broadcast from Server to Instrumentalists

2 Instrumentalist(s) Tick received: 7 [SERVER] TICK 8 sent to 2 Instrumentalist(s) Tick received: 8 [SERVER] TICK 9 sent to 2 Instrumentalist(s) Tick received: 9 [SERVER] TICK 10 sent to 0 Instrumentalist(s) Tick received: 10 [SERVER] TICK 0 sent to 1 Instrumentalist(s) Tick received: 0 Tick received: 1 [SERVER] TICK 1 sent to 1 Instrumentalist(s) [SERVER] TICK 2 sent to 1 Instrumentalist(s) Tick received: 2 [SERVER] TICK 3 sent to 1 Instrumentalist(s) Tick received: 3 [SERVER] TICK 4 sent to 2 Instrumentalist(s) Tick received: 4 	8. Back Enter Number to Proceed: 6 --- twinkle --- 1. View Part List 2. Create Part 3. View Part 4. Edit Part (Append Note) 5. Delete Part 6. Start Rehearsal 7. Stop Rehearsal 8. Back Enter Number to Proceed: 6 --- twinkle --- 1. View Part List 2. Create Part 3. View Part 4. Edit Part (Append Note) 5. Delete Part 6. Start Rehearsal 7. Stop Rehearsal 8. Back Enter Number to Proceed:	[TICK 1]: C4 [TICK 2]: C4 [TICK 3]: G4 [TICK 4]: G4 [TICK 5]: A4 [TICK 6]: A4 [TICK 7]: G4 Rehearsal done --- twinkle --- 1. View Part List 2. View Part 3. Edit Part (Append Note) 4. Join Rehearsal 5. Back Enter Number to Proceed: 4 Parts to Choose From: piano.txt violin.txt Enter Part Name: piano.txt Ready. [TICK 4]: G4	Ready. [TICK 1]: C4 [TICK 3]: E4 [TICK 5]: F4 [TICK 7]: C4 Rehearsal done --- twinkle --- 1. View Part List 2. View Part 3. Edit Part (Append Note) 4. Join Rehearsal 5. Back Enter Number to Proceed: 4 Parts to Choose From: piano.txt violin.txt Enter Part Name: violin.txt Ready. [TICK 1]: C4 [TICK 3]: E4
--	---	---	--

Figure 5: A client joining mid-rehearsal synchronises with the current tick and executes accordingly

CHALLENGES FACED & SOLUTIONS

1) BLOCKING SOCKET BEHAVIOUR

- Initially, the server used a blocking `accept()` call, which prevented the system from performing other operations like broadcasting ticks.
- A transition to non-blocking sockets, at first, introduced further issues where the server would skip execution when no connections were present.
- These were resolved by restructuring the main loop to continuously process both incoming connections and system logic.

2) HANDLING CLIENT DISCONNECTS

- When clients disconnected at the end of a rehearsal, the server was terminated due to a broken pipe exception.
- This was resolved by ignoring the `SIGPIPE` signal and implementing logic to remove disconnected instrumentalists from the array of active users, allowing for execution to continue without interruption.

3) FILE ACCESS DESIGN

- The initial design had local file access on each client and the server too, breaking the distributed model during a performance.
- This required a redesign, where the server became the source of truth, sending part files to clients during the join operation.
- The change ensured that the system function aligned more towards a true client-server architecture.

4) INTEGRATING IPC WITHOUT DISRUPTION

- Introducing inter-process communication posed a challenge as the existing architecture was single-threaded.
- Instead of a redesign, a minimal approach was adopted using a pipe between parent and child processes, demonstrating IPC without affecting core functionality.